Rapport du projet de Deep Learning

Fatoumata NDOUR, Valentin ABONGO

March 2026

1 Introduction

La génération de données constitue un domaine important de l'apprentissage automatique, avec des applications telles que la synthèse d'images ou l'augmentation de données. Parmi les approches existantes, les *Generative Adversarial Networks* (GANs) ont permis de générer des données réalistes, mais présentent des problèmes de stabilité et de diversité.

Les *Wasserstein GANs* (WGANs) ont été introduits afin d'améliorer ces limitations en s'appuyant sur la distance de Wasserstein, offrant ainsi un apprentissage plus stable. Plus récemment, les *Score-Based Generative Models* (SGMs) proposent une approche alternative basée sur un processus de débruitage progressif.

Dans ce travail, nous comparons ces deux approches dans le cadre de la génération d'images, en les entraînant sur un jeu de données standard. Cette étude vise à analyser leur capacité à capturer la structure des données et à générer des échantillons réalistes et diversifiés.

L'objectif est d'analyser leur capacité à générer des échantillons représentatifs des données réelles, à travers des évaluations qualitatives et quantitatives.

Le reste du rapport est organisé comme suit. La section suivante présente les méthodes étudiées, suivie de la description des jeux de données, de l'implémentation, des résultats expérimentaux et de leur analyse.

2 Méthodes

2.1 Présentation de la tâche

L'objectif de ce travail est de comparer deux approches de génération de données, à savoir les Wasserstein Generative Adversarial Networks (WGANs) et les Score-Based Generative Models (SGMs), dans le cadre de la génération d'images. Pour cela, les deux modèles sont entraînés sur différents jeux de données d'images, afin d'évaluer leur capacité à approximer des distributions de nature variée. En particulier, l'étude repose à la fois sur un jeu de données réel d'images (*FashionMNIST*) et sur un jeu de données synthétique représentant des structures géométriques simples (cercles), permettant d'analyser le comportement des

modèles sur des distributions plus contraintes. Chaque modèle apprend à approximer la distribution des données réelles et à générer de nouveaux échantillons similaires. La comparaison repose sur l'analyse de la qualité des images générées, de leur diversité, ainsi que de la stabilité de l'apprentissage. Cette évaluation est réalisée à l'aide de critères qualitatifs, notamment par visualisation des images produites, ainsi que par une analyse des fonctions de perte et de la capacité des modèles à reproduire la structure globale des données. L'objectif final est de mettre en évidence les forces et les limites de chaque approche dans différents contextes de génération d'images.

2.2 Wasserstein Generative Adversarial Networks (WGAN)

2.2.1 Contexte et rappel des GANs

Les Generative Adversarial Networks (GANs) reposent sur un cadre adversarial entre deux modèles paramétrés :

- un générateur G_θ de paramètres θ, qui transforme un bruit $z \sim p_z$ en un échantillon $G_\theta(z)$,
- un discriminateur D_ω de paramètres ω, qui estime la probabilité qu'un échantillon provienne des données réelles.

Le problème d'optimisation s'écrit :

$$\min_\theta \max_\omega \mathbb{E}_{x\sim p_r}[\log D_\omega(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D_\omega(G_\theta(z)))]$$

On note p_r la distribution des données réelles (échantillons réels) et p_g la distribution des données générées par le modèle, définie comme la distribution de $G_\theta(z)$ lorsque $z \sim p_z$.

Ce cadre correspond à la minimisation de la divergence de Jensen-Shannon entre p_r et p_g.

2.2.2 Distances entre distributions

Plusieurs distances peuvent être utilisées pour comparer deux distributions p_r et p_g :

- **Distance de variation totale**

$$\delta(p_r, p_g) = \sup_A |p_r(A) - p_g(A)|$$

- **Divergence de Kullback-Leibler**

$$D_{KL}(p_r|p_g) = \int p_r(x) \log \frac{p_r(x)}{p_g(x)} dx$$

- **Divergence de Jensen-Shannon**

$$D_{JS}(p_r|p_g) = \frac{1}{2}D_{KL}(p_r|m) + \frac{1}{2}D_{KL}(p_g|m)$$

avec $m = \frac{1}{2}(p_r + p_g)$

- **Distance de Wasserstein (forme primaire)**

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|]$$

où $\Pi(p_r, p_g)$ est l'ensemble des couplages de p_r et p_g.

2.2.3 Limites des GANs classiques

Lorsque les supports de p_r et p_g ne se recouvrent pas, les divergences comme D_{JS} deviennent constantes, ce qui entraîne un gradient nul :

$$\nabla_\theta D_{JS}(p_r|p_g) \approx 0$$

Cela provoque :

- une instabilité de l'apprentissage,
- des gradients peu informatifs,
- le phénomène de *mode collapse*.

2.2.4 Introduction des WGAN

Les WGANs remplacent ces divergences par la distance de Wasserstein, qui reste informative même lorsque les distributions sont disjointes.

En utilisant la dualité de Kantorovich-Rubinstein, la distance devient :

$$W(p_r, p_g) = \sup_{\|D_\omega\|_L \leq 1} \mathbb{E}_{x\sim p_r}[D_\omega(x)] - \mathbb{E}_{z\sim p_z}[D_\omega(G_\theta(z))]$$

Le discriminateur devient alors un *critique* $D_\omega : \mathbb{R}^d \to \mathbb{R}$.

2.2.5 Condition de Lipschitz

La contrainte $\|D_\omega\|_L \leq 1$ garantit que D_ω est 1-Lipschitz.

Elle est imposée en pratique par :

- **Weight clipping** :

$$\omega \leftarrow \text{clip}(\omega, -c, c)$$

où c est un hyperparamètre,

- **Gradient penalty** :

$$\lambda \mathbb{E}_{\hat{x}} \left[(\|\nabla_{\hat{x}} D_\omega(\hat{x})\|_2 - 1)^2 \right]$$

2.2.6 Fonction objectif

Le problème devient :

$$\min_{\theta} \max_{\omega \in \mathcal{W}} \mathbb{E}_{x \sim p_r}[D_\omega(x)] - \mathbb{E}_{z \sim p_z}[D_\omega(G_\theta(z))]$$

Cette quantité joue à la fois le rôle de distance et de fonction de perte.

2.2.7 Algorithme et hyperparamètres

L'apprentissage suit un schéma itératif :

- nombre d'itérations du critique : n_{critic}
- taux d'apprentissage : α
- taille du batch : m
- paramètre de clipping : c
- coefficient du gradient penalty : λ

À chaque itération :

- mettre à jour ω (critique) n_{critic} fois,
- puis mettre à jour θ (générateur).

2.2.8 Avantages

- gradients informatifs même pour distributions disjointes,
- meilleure stabilité d'apprentissage,
- interprétation géométrique claire (transport optimal).

2.2.9 Limites

- *mode collapse* toujours possible,
- dépendance à la contrainte de Lipschitz,
- difficulté à capturer des structures fines.

2.3 Score-Based Generative Models (SGMs)

2.3.1 Idée globale

On considère $Z_1, Z_2, ..., Z_n$, n échantillons de $\mathbb{R}^d$ générés indépendamment selon une loi de probabilité de densité $p_0(x)$. À partir des échantillons, l'objectif est de pouvoir générer des données qui simulent la loi p_0.

La méthode SGM opère schématiquement ainsi: elle consiste à être capable de bruiter des données qui suivent la loi p_0 de manière à obtenir une version bruitée suivant une loi plus facile à simuler de densité p_T (comme une gaussienne) puis d'en déduire une méthode pour débruiter une donnée de loi p_T afin de revenir sur une simulation de la loi initiale p_0.

La difficulté n'est pas tant le choix de bruitage mais trouver une méthode de débruitage des données. Une fois trouvée, pour simuler la loi initiale, il suffit de simuler la loi simple p_T puis d'appliquer la méthode de débruitage.

La méthode SGM se fonde sur la notion de score: pour p une densité de probabilité sur $\mathbb{R}^d$, on appelle score de p (s'il existe) le gradient de $\log \circ p$, soit $\nabla_x \log(p(x))$.

2.3.2 SGM via SDE

La méthode de bruitage sera l'application d'un bruit décrit par une équation différentielle stochastique (ou en anglais Stochastic Differential Equation (SDE)). Plus exactement on décrit le processus de bruitage comme un processus $(X_t)_{t\in[0,T]}$ à valeurs sur $\mathbb{R}^d$ où X_0 est de loi p_0, X_T est de loi p_T et le processus vérifie la SDE suivante en forward:

$$d\mathbf{X_t} = f(\mathbf{X_t}, t)dt + g(t)d\mathbf{W}_t$$

où :

- $f : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ est le coefficient de drift
- $g : [0,T] \to \mathbb{R}$ est le coefficient de diffusion
- $d\mathbf{W}_t$ est l'incrément du processus de Wiener sur $\mathbb{R}^d$ (bruit blanc)

On suppose que pour tout $t \in [0,T]$, X_t admet une densité p_t. De plus on admet que X_t sachant X_0 peut se décrire à l'aide d'une densité $p_{0t}(x_t|x_0)$.

On peut définir le processus de débruitage comme étant le processus $(X_t)_{t\in[0,T]}$ mais vu en remontant le temps depuis T. La théorie montre l'existence d'une SDE (E) décrivant le débruitage:

$$(E) \quad d\mathbf{X_t} = [f(\mathbf{X_t}, t) - g^2(t)\nabla_{\mathbf{X_t}} \log p_t(\mathbf{X_t})]dt + g(t)d\bar{\mathbf{W}}_t$$

où:

- f et g sont les mêmes que précédemment

- $d\bar{\mathbf{W}}_t$ est l'incrément du processus de Wiener sur $\mathbb{R}^d$ (dans la direction du temps inversé)
- dt est un pas de temps négatif

On observe ici si on connaît f, g, que l'on sait approcher les scores $\nabla \log p_t$ et que l'on sait simuler une SDE alors on est en capacité de débruiter les données et donc de simuler p_0. Pour le détail, simuler numériquement un processus stochastique suivant une SDE est possible grâce à des méthodes de discrétisation comme celle d'Euler-Maruyama ou encore celles présentes dans [4] à la page 19 (reverse diffusion sampling) et à la page 22 (Predictor-Corrector (PC) sampling).

2.3.3 Estimation des fonctions scores

Le point fondamental de la méthode SGM est l'estimation des fonctions $\nabla \log p_t$. Pour ce faire, on utilise un réseau de neurones $s_\theta(x,t)$, où θ désigne les paramètres du réseau et qui approxime $\nabla \log p_t(x)$. Lors de l'entraînment, le but sera de minimiser la fonction coût suivante:

$$\min_\theta \mathbb{E}_{t\sim\mathcal{U}(0,T)}[\lambda(t)\mathbb{E}_{X_0\sim p_0}\mathbb{E}_{X_t\sim p_{0t}(X_t|X_0)}[\|s_\theta(X_t,t)-\nabla_{X_t}\log p_{0t}(X_t \mid X_0)\|_2^2]],$$

où $\mathcal{U}(0,T)$ est la loi uniforme sur $[0,T]$, $p_{0t}(x_t \mid x_0)$ décrit la probabilité de transition de x_0 à x_t, et $\lambda(t) \in \mathbb{R}_{>0}$ une fonction de pondération positive, comme par exemple $g(t)^2$. Cette fonction coût laisse bien entendre que pour la minimisation, on doit être capable de simuler le processus de bruitage où $X_0 \sim p_0$.

2.3.4 Différents types de bruitage

Dans la littérature, 3 types de bruitage sont privilégiés. Ces bruitages ont une équation stochastique en forward de la forme:

$$d\mathbf{X_t} = a(t).\mathbf{X_t}dt + g(t)d\mathbf{W}_t$$

Sous cette forme, on obtient que X_t sachant X_0 est une gaussienne de moyenne $\alpha(t).X_0$ où

$$\alpha(t) = \exp(\int_0^t a(s)\,ds)$$

et de matrice de covariance $\sigma^2(t).I_d$ où

$$\sigma^2(t) = \int_0^t g(s)^2 \exp\left(2\int_t^s a(u)\,du\right)ds)$$

Une autre conséquence est l'égalité suivante:

$$\nabla_{X_t} \log p_{0t}(X_t \mid X_0) = -\frac{X_t - \alpha(t).X_0}{\sigma^2(t)}$$

En raison de cette formule et pour des raisons de stabilité de calculs, l'estimation de cette fonction score peut être remplacée lors de l'apprentissage par l'estimation du bruit $X_t - \alpha(t).X_0 = \epsilon_t \sim \mathcal{N}(0, \sigma^2(t)|X_0)$ par un réseau de neurones $\epsilon_\theta(x_t, t)$ de type U-net.

Pour la suite, les mêmes notations f et g pour désigner les coefficients de drift et de diffusion seront utilisées.

Variance Exploding (VE) SDE

- $f(x,t) = 0$ (pas de drift)
- $g(t) = \sqrt{\frac{d\sigma^2}{dt}(t)}$

Ici $\sigma(t)$ est une fonction qui influe sur la variance dans le processus de bruitage. On suppose au moins σ nulle en 0, dérivable et croissante sur $[0, T]$.
Exemple:

$$\sigma(t) = \sigma_{min}.\left(\frac{\sigma_{max}}{\sigma_{min}}\right)^t$$

Variance Preserving (VP) SDE

- $f(x,t) = -\frac{1}{2}\beta(t).x$
- $g(t) = \sqrt{\beta(t)}$

$\beta(t)$ est à voir comme un niveau de bruit (nommé également en anglais schedule) qui influe sur la variance du processus de bruitage de manière à ce que l'on puisse rendre X_t pour des temps t suffisamment grands proche d'une gaussienne centrée réduite de $\mathbb{R}^d$.
Exemple:

$$\beta(t) = \beta_{min} + (\beta_{max} - \beta_{min}).t$$

Sub-VP SDE

- $f(x,t) = -\frac{1}{2}\beta(t).x$
- $g(t) = \sqrt{\beta(t)\exp(1 - 2.\int_0^t \beta(s)\,ds)}$

Même $\beta(t)$ que pour Variance Preserving (VP).

2.3.5 Débruitage déterministe : Probability Flow ODE

Pour simuler $X_0 \sim p_0$, il est possible de considérer un autre processus de débruitage que celui défini par la SDE (E).
On introduit à la place l'ODE suivante :

$$(E') \quad dX_t = \left[f(X_t, t) - \frac{g^2(t)}{2} \nabla_{X_t} \log p_t(X_t) \right] dt$$

Cette équation est appelée *probability flow ODE*. Elle a la propriété remarquable que les marginales (p_t) des solutions coïncident avec celles du processus de diffusion (forward ou reverse), bien que les trajectoires soient déterministes.

Ainsi, en approximant le score $\nabla \log p_t$, on peut simuler $X_0 \sim p_0$ en échantillonnant $x_T \sim p_T$, puis en résolvant numériquement cette ODE de T vers 0 avec des méthodes numériques comme celle de Runge-Kutta.

3 Jeux de données

Dans ce travail, les jeux de données sont choisis de manière à mettre en évidence les différences de comportement entre les WGANs et les Score-Based Generative Models (SGMs). Ce choix est directement lié aux propriétés théoriques et aux limitations des modèles présentées dans la section précédente.

Afin d'évaluer les modèles dans des contextes complémentaires, nous considérons à la fois un jeu de données d'images réalistes (*Fashion-MNIST*) et un jeu de données synthétique basé sur des structures géométriques simples (cercles). Ce choix permet d'analyser la capacité des modèles à apprendre des distributions variées, allant de données visuelles riches à des structures plus contraintes.

3.1 Description des datasets

Fashion-MNIST est un jeu de données composé d'images en niveaux de gris de taille 28×28, représentant différentes catégories d'articles vestimentaires, telles que des chaussures, des sandales, des sacs ou des vêtements.

Chaque image est associée à une classe, mais dans ce travail, les modèles sont entraînés de manière non supervisée, sans utiliser ces labels.

En complément, un jeu de données synthétique composé d'images de cercles a également été considéré. Ces images, également de taille 28×28, représentent des formes circulaires de tailles et de positions variables. Cette base de données contient 10000 images, 8000 pour l'entraînement et 2000 comme ensemble de validation lors de l'entraînement. Ce jeu de données permet d'étudier le comportement des modèles sur une structure géométrique simple et fortement contrainte, où l'information utile est concentrée sur une forme précise. Il constitue ainsi un cadre pertinent pour analyser la capacité des modèles à apprendre des distributions présentant une structure spatiale globale.

3.2 Objectif du choix des jeux de données

Les jeux de données retenus ont été choisis de manière à évaluer plusieurs propriétés essentielles des modèles génératifs.

Le dataset *Fashion-MNIST* permet d'étudier la capacité des modèles à apprendre une distribution multimodale, chaque catégorie d'objet (chaussures, sacs, vêtements, etc.) correspondant à un mode distinct. Il constitue également un cadre pertinent pour analyser la qualité visuelle des images générées ainsi que la reproduction de structures visuelles fines.

Le jeu de données synthétique des cercles permet quant à lui d'évaluer la capacité des modèles à apprendre une structure géométrique globale et fortement contrainte. Il est particulièrement adapté pour analyser le comportement des modèles sur des distributions présentant une organisation spatiale simple mais rigide.

L'utilisation conjointe de ces deux jeux de données permet ainsi de comparer les performances des WGANs et des SGMs sur des distributions de nature différente.

4 Détails de l'implémentation

4.1 WGANs

Les expériences menées dans ce travail reposent sur le même protocole d'implémentation pour les deux jeux de données considérés : *Fashion-MNIST* et le jeu de données synthétique des cercles.

Les données sont d'abord prétraitées afin d'être compatibles avec le modèle. Dans le cas de *Fashion-MNIST*, le dataset contient 60 000 images d'entraînement et 10 000 images de test. Les images, de taille 28×28, sont converties en tenseurs et normalisées dans l'intervalle $[-1, 1]$, en cohérence avec la fonction d'activation *tanh* utilisée en sortie du générateur. Elles sont ensuite regroupées en mini-lots de taille 64.

L'architecture implémentée repose sur un réseau convolutionnel inspiré de la documentation et de notebooks de référence, puis adaptée à notre cadre expérimental. Le choix d'une architecture de type CNN s'explique par sa capacité à capturer efficacement les structures spatiales et les motifs visuels présents dans les images.

Le générateur repose sur des couches de convolution transposée (*ConvTranspose2D*), associées à une normalisation par batch et à des activations $ReLU$, avec une fonction *tanh* en sortie. Le critique utilise des couches convolutionnelles classiques suivies d'activations *LeakyReLU*. Cette structure permet d'extraire et de reconstruire progressivement les caractéristiques visuelles des données.

Le vecteur de bruit d'entrée, de dimension 100, est échantillonné suivant une distribution gaussienne et constitue l'entrée du générateur pour la production de nouveaux échantillons synthétiques.

L'optimisation repose sur l'algorithme *RMSProp*. L'entraînement suit une procédure adversariale alternée dans laquelle le critique est mis à jour cinq

fois ($n_critic = 5$) avant chaque mise à jour du générateur, afin d'obtenir une estimation plus stable de la distance de Wasserstein. La contrainte de Lipschitz est imposée par un *weight clipping* fixé à 0.01.

Le choix final des hyperparamètres n'a pas été figé dès le départ. Plusieurs essais ont été réalisés afin d'évaluer l'impact du taux d'apprentissage, de la taille des batchs, du facteur de clipping et du nombre d'époques sur la qualité des images générées. Certains ajustements ont permis d'accélérer l'apprentissage, mais au prix d'une instabilité plus marquée ou d'une dégradation visuelle des résultats. Après plusieurs expérimentations, la configuration offrant le meilleur compromis entre stabilité et qualité visuelle a été retenue : un taux d'apprentissage de 5×10^{-5}, une taille de batch de 64 et un entraînement arrêté à 30 époques.

Le critère d'arrêt a été défini à partir de l'évaluation qualitative des images générées. Il a été observé qu'au-delà de 30 à 35 époques, la qualité visuelle n'évoluait plus positivement et tendait à se dégrader. Le point d'arrêt a donc été fixé à 30 époques, correspondant au meilleur compromis observé.

Afin de suivre la dynamique d'apprentissage, une fonction dédiée a été mise en place pour capturer les images générées à intervalles réguliers. Cette approche permet d'observer l'évolution progressive des échantillons produits et la capacité du modèle à apprendre la distribution des données.

Le temps total d'apprentissage avec la configuration finale est d'environ 35 minutes.

À la fin de l'entraînement, les fonctions de perte du générateur et du critique sont récupérées afin d'analyser la stabilité du processus d'apprentissage. Enfin, un ensemble de 64 vecteurs de bruit aléatoire est utilisé pour générer 64 images synthétiques, servant de base à la comparaison visuelle avec les données réelles.

4.2 SGMs

Pour manipuler les SGMs, le package python Torchdiff a été utilisé. Pour appréhender cette bibliothèque, on s'est inspiré en plus de [8] des références [2], et [7]. Il est intéressant d'évoquer que d'autres approches été possibles comme il a pu être constaté dans la référence [3].

La méthode SGM pour être utilisé nécessite de faire des choix. D'abord le choix du réseau de neurones qui apprend la fonction score/bruit au fil du temps: un CNN de type U-net a été choisi pour apprendre le bruit dont on déduit le score. Dans l'implémentation de torchdiff, ce réseau se veut un peu plus complexe car il contient également dans son architecture un mécanisme d'attention qui permet de faire de la génération conditionnellement à un texte converti en embedding, ce qui permet au réseau d'estimer une fonction de score sachant l'embedding. Ainsi il est possible de générer une image à partir d'un texte. Cependant cette propriété du réseau ne sera pas abordé. De plus, l'optimiseur utilisé pour la backpropagation a été AdamW de pyTorch où juste le paramètre lr a été mis à 10^{-4}.
Le type de bruitage choisi est Variance Preserving (VP) vu plus haut. La fonction β (schedule) est la même fonction (dite "linear" dans torchdiff) que celle donnée en exemple dans la partie Variance Preserving (VP) SDE. Pour simuler

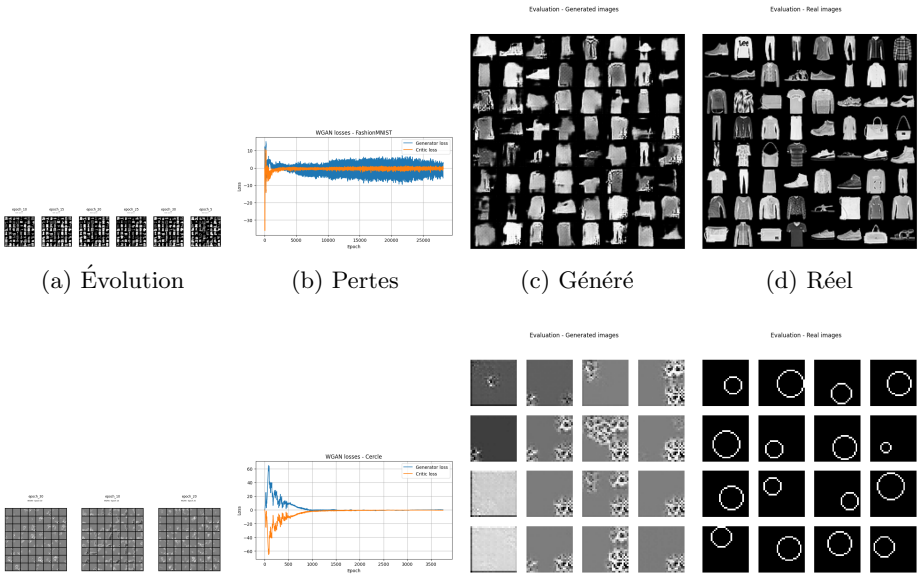
la SDE de bruitage, la formule $X_t - \alpha(t).X_0 = \epsilon_t \sim \mathcal{N}(0, \sigma^2(t)|X_0)$ donne un moyen explicite de simuler le bruitage puisque X_0 est connu, de même que $\alpha(t)$ et $\sigma(t)$. Pour le débruitage, la documentation torchdiff confirme l'utilisation de la méthode de discrétisation d'Euler-Maruyama pour simuler la SDE reverse. Une fois tous ces choix effectuées, il est possible de faire apprendre au réseau de neurones U-net le bruit. À la fin de l'apprentissage, il suffit d'envoyer le U-net à un échantillonneur qui simulera la SDE reverse à partir d'un échantillon tiré selon la loi p_T (T vaut 1 dans l'implémentation).

Pour la base de données FMNIST, le nombre d'époques pour l'apprentissage de la fonction score a pu monter jusqu'aux environs de 10 tandis que pour celle des cercles, c'est allé jusqu'à presque 50 (il y a eu early stopping). Cette différence est la richesse de la base FMNIST. À cela, il faut souligner la nécessité d'utiliser des GPU et non des CPU. Les calculs se veulent particulièrement coûteux lors de l'apprentissage et dans une moindre mesure aussi pour la génération des images où la discrétisation de l'intervalle $[0, 1]$ pour la discrétisation d'Euler-Maruyama va jusqu'à 1000.

5 Résultats et Analyse

5.1 Résultats WGANs

À l'issue de l'entraînement du WGAN, les résultats obtenus sont présentés sous forme de grilles d'images et de courbes d'apprentissage. Nous présentons les fonctions de perte, l'évolution progressive des échantillons générés au cours de l'entraînement, ainsi qu'une comparaison visuelle entre les données réelles et les données synthétiques produites.



(a) Évolution (b) Pertes (c) Généré (d) Réel

(e) Évolution (f) Pertes (g) Généré (h) Réel

Figure 1: Résultats obtenus avec le WGAN sur les deux jeux de données : Fashion-MNIST (ligne supérieure) et cercles (ligne inférieure). Les colonnes présentent respectivement l'évolution de l'apprentissage, les fonctions de perte, les échantillons générés et les données réelles.

5.2 Analyse des résultats du WGAN

Les résultats obtenus montrent que le WGAN parvient à apprendre la structure globale des données, avec une qualité variable selon le jeu de données.

Dans le cas de *Fashion-MNIST*, les images générées s'améliorent progressivement au cours de l'entraînement. À partir d'environ 10 époques, les formes commencent à apparaître, puis deviennent plus lisibles vers 20 à 30 époques. Au-delà, l'amélioration reste limitée, ce qui justifie le choix du critère d'arrêt.

La comparaison avec les images réelles montre que le modèle capture correctement la forme générale des objets, tels que les vêtements et les chaussures. En revanche, les détails fins, comme les motifs, les lacets, les talons ou certaines décorations, ne sont pas correctement reproduits. Le modèle apprend donc principalement les contours et la structure globale des différentes classes.

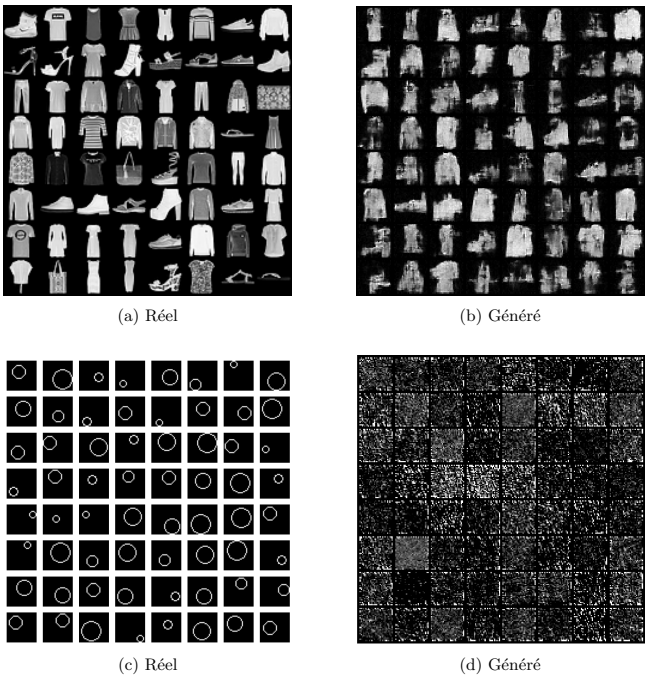
Par ailleurs, certaines catégories, comme les *tops* et les *trousers*, semblent plus fréquemment générées que d'autres, ce qui suggère une tendance à un *mode collapse* partiel. Les courbes de perte montrent une forte variation au début de l'apprentissage, suivie d'une stabilisation autour de zéro, traduisant un apprentissage adversarial globalement stable.

En revanche, sur le jeu de données synthétique des cercles, le modèle ne parvient pas à reconstruire correctement la structure géométrique attendue.

Les images générées restent bruitées et aucune forme circulaire nette n'émerge, même après plusieurs époques. Les courbes de perte atteignent rapidement un plateau, sans amélioration visuelle significative.

Ces résultats mettent en évidence que le WGAN classique est plus adapté à l'apprentissage de la structure globale d'images multimodales qu'à la modélisation de données définies sur une sous-variété géométrique simple.

5.3 Résultats de SGMs



(a) Réel (b) Généré

(c) Réel (d) Généré

Figure 2: Résultats obtenus avec SGM sur les deux jeux de données : Fashion-MNIST (ligne supérieure) et cercles (ligne inférieure). Chaque ligne permet de comparer les données réelles et générées d'une même base de données

On observe pour la base F-MNIST que les images générées ont capturées la structure des images réelles utilisées lors de l'entraînement. Cependant le détail des images n'a pas été bien capturé. On observe également une certaine variété des images générées, ce qui laisse entendre l'absence de mode collapse.

Pour la base des cercles, les données générées ne réussissent pleinement pas à saisir les cercles des images réelles. Sachant que l'apprentissage s'est arrêté par un early stopping lors des expérimentations, on peut conclure que le modèle SGM tel qu'il est n'est pas adapté à ce type d'images.

6 Conclusion

L'analyse comparative des résultats met en évidence des comportements différents entre le WGAN et le SGM selon la nature des données. Sur *Fashion-MNIST*, les deux modèles parviennent à capturer la structure globale des images réelles. Le WGAN reproduit correctement les contours et la forme générale des objets, mais présente une perte de détails fins et une légère tendance à privilégier certaines catégories. Le SGM, quant à lui, semble générer une plus grande diversité d'images, suggérant une meilleure couverture des modes de la distribution.

En revanche, sur le jeu de données synthétique des cercles, aucun des deux modèles ne parvient à reconstruire correctement la structure géométrique attendue. Le WGAN produit des résultats bruités sans forme circulaire nette, tandis que le SGM ne réussit pas non plus à capturer la sous-variété définie par les cercles, malgré les expérimentations réalisées.

Ainsi, les deux approches montrent de bonnes capacités à apprendre la structure globale d'images complexes, mais révèlent des limites importantes dès lors qu'il s'agit de modéliser des structures géométriques simples et contraintes.

Bibliographie

[1] https://medium.com/@samaniloqman91/torchdiff-a-pytorch-library-for-denoising-diffusion-probabilistic-models-and-beyond-d15156e9e3b5 (Introduction DDPM)

[2] https://medium.com/@samaniloqman91/score-based-generative-modeling-with-sdes-theory-and-implementation-in-torchdiff-db9a20a84978 (Introduction SGM via SDE)

[3] https://colab.research.google.com/drive/120kYYBOVa1i0TD85RjlEkFjaWDxSFUx3?usp=sharing#scrollTo=XCR6m0HjWGVV (Code sur SGM via SDE)

[4] https://arxiv.org/pdf/2011.13456 (article de Song)

[5] https://arxiv.org/abs/2006.11239 (Linear schedule)

[6] https://arxiv.org/pdf/2102.09672 (Cosinus schedule)

[7] https://github.com/LoqmanSamani/TorchDiff/blob/systembiology/examples/sde/vp-sde/vp_sde.ipynb (Code pour implémenter la méthode SGM)

[8] https://torchdiff.readthedocs.io/en/latest/sde.html (torchdiff)

[9] https://arxiv.org/pdf/1511.06434

[10] https://arxiv.org/pdf/1701.04862

[11] https://arxiv.org/pdf/1611.02163

[12] https://github.com/kpandey008/wasserstein-gans/blob/master/WGAN.ipynb

[13] https://www.geeksforgeeks.org/deep-learning/wasserstein-generative-adversarial-networks-wgans-convergence-and-optimization/

[14] https://github.com/eriklindernoren/PyTorch-GAN/tree/master/implementations/wgan (Ancienne implémentation du GAN)